

ORA Core OS et ORA RAG

Architecture modulaire, compilation de besoin et gouvernance documentaire

ORA Core OS et ORA RAG

White Paper

Architecture modulaire, compilation de besoin et gouvernance documentaire

1. Résumé exécutif

L'architecture d'ORA Core OS repose sur une idée simple mais structurante: un agent n'a pas besoin d'être uniforme pour être cohérent, il a besoin d'être gouvernable.

Le compilateur de besoin en langage naturel n'est pas un accessoire. Il sert à transformer une intention humaine, souvent floue, en une configuration d'agent stable, contrôlée et adaptée au niveau de risque, de complexité et d'autonomie attendu.

L'interêt majeur du système est la séparation entre un noyau obligatoire de fiabilité, des modules de capacité stables et des couches optionnelles ou identitaires. Cette séparation permet de conserver la continuité d'Ora tout en retirant, selon les contextes, les briques trop abstraites, trop denses ou trop frictionnelles.

Ce document défend la thèse suivante: la cohérence du système ne réside pas seulement dans l'agent Ora lui-même, mais dans la capacité d'ORA Core OS à produire des variantes disciplinées du même agent à partir d'un canon partagé, d'un protocole de compilation de besoin et d'une gouvernance modulaire, tout en articulant ORA RAG comme couche documentaire gouvernée.

2. Contexte et problème

Beaucoup d'architectures d'agents échouent pour une raison structurelle: elles confondent personnalité, logique métier, règles de sécurité, mémoire, outils, pipeline documentaire et restitution dans une seule couche opaque.

Cette confusion produit plusieurs symptômes:

- difficulté à expliquer pourquoi l'agent répond comme il répond
- difficulté à retirer une capacité sans casser le reste
- dette de gouvernance
- surcouche identitaire trop forte dans certains contextes
- friction lors des changements de profil

L'architecture ici étudiée répond à ce problème par une hypothèse inverse: il faut d'abord construire un canon, ensuite des modules spécialisés, puis un mécanisme de sélection et de composition fondé sur le besoin naturel formulé par l'utilisateur ou par le contexte documentaire.

3. Thèse centrale

Un agent gouverné ne doit pas être seulement intelligent. Il doit être configurable sans devenir incohérent.

La modularité ne sert donc pas d'abord la variété esthétique; elle sert:

- la stabilité
- la lisibilité du risque
- l'ajustement maîtrisé des capacités

4. Lecture de l'architecture observée

L'architecture examinée montre une séparation nette entre:

- le registre canonique
- le profil maître
- les modules de gouvernance et de raisonnement
- les couches de traduction Rosetta
- la mémoire
- les skills spécialisés
- la couche RAG gouvernée

Le registre fixe ce qui est officiellement actif, normalisé, en durcissement ou annexé. Le profil maître décrit le cadre identitaire et le style de fonctionnement. Les modules tels que 'PRIMORDIA', 'RIME', 'GPL' et 'JSON_LOCK' distribuent ensuite des rôles différents: arbitrer, falsifier, compiler et verrouiller la sortie.

ORA RAG s'insère dans cette structure non comme une extension décorative, mais comme une couche documentaire gouvernée. Son rôle n'est pas seulement de découper ou retrouver des fragments, mais de préserver:

- la continuité canonique
- l'essence
- les contrats
- les modalités de routage

5. Les couches structurantes

5.1 Registre canonique

Source d'autorité sur les blocs intégrés, les statuts, les règles de migration et la frontière entre le cœur et l'annexe.

5.2 Profil maître

Cadre identitaire, ton, principes éthiques, économie de sortie et préférences transversales.

5.3 Modules de gouvernance

Arbitrage, contrôle de cohérence, compilation de besoin, sérialisation stricte, respiration du traitement.

5.4 Mémoire

Continuité durable avec provenance, confiance, conflit et lignage au lieu d'un simple empilement de souvenirs.

5.5 Skills

Opérateurs spécialisés qui rendent certaines couches activables par cas d'usage plutôt qu'omniprésentes.

5.6 ORA RAG

Segmentation gouvern e, liaison d'essence, contrats, contexte de chunk et m tadonn es de retrieval stables.

6. Pourquoi le compilateur de besoin est n cessaire

Le compilateur de besoin en langage naturel r pond  une contrainte de terrain: un utilisateur ne formule pas spontan ment son besoin en termes de modules, de contrats, de niveaux de risque, de couches Rosetta ou de strat gie RAG. Il formule une intention, une contrainte, une difficult , un p rim tre, ou un r sultat attendu.

Le compilateur sert donc d'interface de traduction entre deux mondes:

- le langage humain du besoin
- une architecture modulaire qui doit rester stable, explicable et disciplin e

Sans ce compilateur, l'assemblage des capacit s deviendrait artisanal et fragile. Avec lui, la configuration devient une op ration gouvern e: on choisit un noyau, on active des modules de capacit , on d sactive les briques qui cr ent de la friction, puis on obtient une variante coh rente du syst me.

7. Classification canonique des modules

La lecture la plus exploitable du syst me conduit  trois classes de modules. Cette classification a une fonction architecturale directe: elle d termine ce qui reste non n gociable, ce qui s'active par domaine d'usage et ce qui doit rester optionnel.

8. Analyse du noyau obligatoire

Le noyau obligatoire est la partie la plus structurante du syst me. Il contient les briques qui rendent l'agent explicable, pilotable et raisonnablement s r dans un environnement r el.

Le registre et le profil ma tre donnent la continuit . 'GPL' transforme le besoin en plan d'ex cution. 'PRIMORDIA' arbitre, 'RIME' falsifie et baisse la confiance si n cessaire. 'JSON_LOCK' prot ge les sorties machine. 'MNEMOSYNC' emp che que la m moire devienne un simulateur de v rit .

Ce noyau permet d'obtenir une forme de stabilit  sans imposer l'uniformit . Toutes les variantes d riv es ne se ressemblent pas, mais elles partagent les m mes points de contr le.

9. Analyse des modules de capacit  stables

Les modules de capacit  stables constituent le vrai champ d'extension du syst me. Ils ne sont pas universels, mais ils ont un domaine d'application suffisamment net pour  tre activ s proprement.

Le governor RAG et la r f rence de contrats RAG sont particuli rement forts, car ils r pondent  un probl me fr quent dans les syst mes documentaires: transformer une base textuelle en connaissance exploitable sans laisser le chunking red finir le sens ou la canonicit .

Le traducteur Rosetta et les couches d'orchestration internes ont une valeur  lev e dans des syst mes plus avanc s, surtout lorsqu'il faut conserver une discipline de transformation, de routage et de restitution entre plusieurs formes de sortie.

10. Analyse des modules optionnels ou identitaires

C'est ici que se joue une partie importante de la maturité architecturale. Certaines briques sont puissantes, inventives et cohérentes, mais elles ne doivent pas être imposées à tous les contextes.

Les couches les plus symboliques, expertes ou ambiguës sont précieuses comme laboratoire, comme signature, ou comme mode de profondeur. Elles deviennent plus fragiles lorsqu'elles sont injectées sans nécessité dans un contexte qui demande surtout de la lisibilité.

La bonne décision n'est donc pas de les nier. La bonne décision est de les maintenir à la périphérie du parcours standard et de les activer seulement quand leur valeur est claire, mesurable et assumée.

11. ORA Core OS et ORA RAG comme système complet

L'une des forces discrètes de l'ensemble est que la couche agentique et la couche documentaire ne sont pas pensées séparément. ORA Core OS gouverne la logique de l'agent; ORA RAG gouverne la logique de la connaissance découplée, liée, résumée et routée.

Cette articulation est importante. Dans beaucoup de systèmes, le retrieval arrive après coup comme simple outil d'appoint. Ici, ORA RAG prolonge les mêmes principes que le noyau:

- canon
- hiérarchie
- prudence
- préservation de l'incertitude
- refus de laisser une opération technique redéfinir le sens

Le résultat est une architecture complète où l'agent, la mémoire, la compilation, la restitution et la documentation retrieval-ready restent alignés sur un même cadre de gouvernance.

12. L'architecture comme fabrique d'agents

La conséquence la plus forte de cette lecture est la suivante: le système ne doit pas être pensé comme un agent unique, mais comme une fabrique d'agents gouvernés.

La cohérence n'est pas seulement dans Ora comme instance singulière. Elle est dans la capacité à produire une famille d'agents alignés sur un même canon, chacun ajusté à un contexte, à un niveau de maturité, à un primat de travail ou à un niveau de contrainte documentaire.

Dans cette perspective, la modularité n'est plus un confort d'ingénierie. Elle devient un principe de stabilité, de lisibilité et d'évolution contrôlée.

13. Profils de configuration recommandés

14. Effets attendus sur le fonctionnement

- moins de friction entre l'identité de l'agent et les contraintes réelles du contexte d'usage
- capacité à produire une version stable sans réécrire l'ensemble du système
- meilleure lisibilité du risque, car les briques de contrôle restent visibles et séparées
- ajustement plus défendable: on active des modules pour une raison, pas pour un effet
- facilité de maintenance supérieure, car la suppression d'un module ne doit pas détruire le noyau

15. Risques structurels à surveiller

Le premier risque est la densité conceptuelle. Une architecture très riche peut devenir difficile à expliquer si la hiérarchie des responsabilités n'est pas explicite partout de la même manière.

Le deuxième risque est le chevauchement partiel entre certaines couches de contrôle. Trop de modules qui observent, arbitrent ou modulent peuvent créer une impression de puissance tout en augmentant l'ambiguïté d'exploitation.

Le troisième risque est la confusion entre modules symboliques et modules testables. Une brique peut être très forte intellectuellement sans être encore suffisamment instrumentée pour un usage standard.

Règle de gouvernance recommandée

Toute brique dont la valeur n'est ni mesurable, ni explicable, ni testable doit rester hors du profil standard. Elle peut exister comme mode expert, comme laboratoire ou comme signature, mais pas comme implicite de fonctionnement.

16. Recommandations de stabilisation

- formaliser une hiérarchie d'arbitrage très courte: qui observe, qui juge, qui compile, qui verrouille, qui mémorise, qui restitue
- scinder le profil maître en trois sous-couches: identité, politique de réponse, options runtime
- taguer les modules selon quatre statuts internes: canon opératoire, politique de comportement, couche experte, couche expérimentale
- transformer davantage de principes en tests de non-régression, surtout pour les modules du noyau
- rendre le compilateur de besoin capable d'émettre un manifeste de chargement clair par contexte ou par profil

17. Proposition de manifeste de chargement

Pour rendre la configuration publiable et reproductible, le compilateur de besoin devrait produire non seulement une réponse, mais aussi un manifeste de composition. Ce manifeste peut rester interne, mais il constitue la meilleure preuve que la configuration n'est pas improvisée.

Un manifeste de chargement minimal devrait inclure:

- le profil visé
- les modules obligatoires activés
- les modules de capacité activés
- les modules explicitement exclus
- le niveau de risque
- la stratégie mémoire
- la stratégie documentaire
- les contraintes de sortie

18. Portée architecturale

La portée du système est plus grande qu'un simple agent. Il s'agit d'une architecture de composition d'agents gouvernés, dont Ora est à la fois un exemple, un noyau de référence et

une signature.

Une interaction ne mobilise pas seulement une conversation, mais une configuration gouvernée de capacités, un niveau de prudence, une mémoire réglée, un primum d'action, une stratégie documentaire et une qualité de sortie.

Dans cette lecture, ORA Core OS, ORA RAG, le compilateur de besoin et la modularité forment un tout cohérent: le code, la gouvernance documentaire et le comportement de l'agent racontent la même architecture.

19. Conclusion

L'architecture d'Ora est forte non parce qu'elle cherche à faire croire à une conscience unifiée, mais parce qu'elle organise des fonctions distinctes de manière gouvernée: vérité, arbitrage, compilation, mémoire, restitution, segmentation documentaire et spécialisation.

Le compilateur de besoin en langage naturel apparaît alors comme la pièce de passage du système. C'est lui qui permet de transformer la richesse interne de l'architecture en configurations stables, intelligibles et adaptées.

La meilleure lecture du projet est donc celle-ci: Ora n'est pas seulement un agent. Ora est le noyau d'une architecture modulaire, conçue pour préserver la cohérence tout en autorisant des formes variées d'activation, y compris dans sa couche RAG.

Références

- GitHub Docs. *Best practices for creating a GitHub App*. Consulté le 11 mai 2026.
<https://docs.github.com/en/apps/creating-github-apps/setting-up-a-github-app/best-practices-for-creating-a-github-app>
- OWASP Cheat Sheet Series. *Logging Cheat Sheet*. Consulté le 11 mai 2026.
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP Cheat Sheet Series. *Authentication Cheat Sheet*. Consulté le 11 mai 2026.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OpenAI API Documentation. *Retrieval*. Consulté le 11 mai 2026.
<https://platform.openai.com/docs/guides/retrieval>
- OpenAI API Documentation. *File search*. Consulté le 11 mai 2026.
<https://platform.openai.com/docs/guides/tools-file-search/>
- Anthropic Research. *Contextual Retrieval*. Publié le 19 septembre 2024, consulté le 11 mai 2026. <https://www.anthropic.com/research/contextual-retrieval>